



OASIS Security Report

CORS Misconfiguration Security Analysis

Date: 2026-06-11 09:52:15 — **Model:** mistral:latest

Vulnerability: CORS Misconfiguration

Project: clean_contracts

Analysis root: ../clean

Table of contents

□ Summary

□ Statistics

□ Files

□ Detailed analysis

□ Assistant

△ Errors & notes

Summary

Analyzed **3** file(s) for CORS Misconfiguration.

[Return to table of contents](#)

Statistics

Files analyzed: 3 **Total findings:** 3

Critical		0
High		0

Medium

2

Low

1

[Return to table of contents](#)

Files

File	Similarity
/work/project/dataset/clean/mycontract_fixed.sol	0.672
/work/project/dataset/clean/simple_dao_fixed.sol	0.627
/work/project/dataset/clean/modifier_reentrancy_fixed.sol	0.621

[Return to table of contents](#)

Detailed analysis

File 1: /work/project/dataset/clean/mycontract_fixed.sol

Similarity score: 0.672

Source lines: 1-31

► Finding 1.1: Vulnerability found (Low)

Vulnerable code

none

Explanation

The provided Solidity contract does not have a Cross-Origin Resource Sharing (CORS) configuration, but this vulnerability is not applicable to Ethereum smart contracts since CORS is an HTTP-level security mechanism.

Impact

Not relevant for this context

Entry point

N/A

Conditions

Not applicable in this context

Remediation

CORS is not relevant for Ethereum smart contracts.

File 2: /work/project/dataset/clean/simple_dao_fixed.sol

Similarity score: 0.627

Source lines: 1-34

► Finding 2.1: Vulnerability found (Medium)

Vulnerable code

```
pragma solidity 0.4.24;  
  
contract SimpleDAO {  
    ...  
}
```

Explanation

The Solidity contract does not have a CORS configuration, which may expose sensitive data if called from another origin.

Impact

Cross-origin data theft or unauthorized API access

Entry point

All functions of the SimpleDAO contract

Execution path

```
...  
SimpleDAO -> Called from another origin without CORS check  
...
```

Exploitation steps

- Call the SimpleDAO contract from another origin
- Interact with the contract (e.g., donate or withdraw functions)

Conditions

Another origin can call the Solidity contract

Remediation

Ensure that the Solidity contract has a proper CORS configuration to restrict calls to specific origins.

File 3: /work/project/dataset/clean/modifier_reentrancy_fixed.sol

Similarity score: 0.621

Source lines: 1-42

► Finding 3.1: Vulnerability found (Medium)

Vulnerable code

```
`supportsToken` hasNoBalance` function call order
```

Explanation

The vulnerable code uses `hasNoBalance` before `supportsToken`, which can potentially allow an attacker to exploit the reentrancy vulnerability.

Impact

Unauthorized token airdrop

Entry point

airDrop function

Execution path

```
```  
1. Attacker calls airDrop() > 2. Contract calls hasNoBalance() (invalid) > 3.
Contract calls supportsToken() > 4. Reentrancy attack occurs````
```

#### Exploitation steps

- Call the airDrop function
- Attacker exploits reentrancy vulnerability by calling into the contract during the hasNoBalance validation

#### Conditions

An attacker can call the airDrop function and manipulate the contract state

#### Remediation

Ensure that `supportsToken` is always evaluated before any action that depends on it in the contract

## Secure example

```
'''
supportsToken > hasNoBalance > ...
'''
```

[Return to table of contents](#)

## Assistant (triage / codebase)

In the OASIS dashboard (JSON report preview), use the chat below to triage findings, exploitation paths, and remediation. Select file, chunk, and finding to narrow context when needed.

[Return to table of contents](#)

## Errors & Notes

No file-level errors were recorded.

[Return to table of contents](#)